Curso de Go (Golang)

En este curso aprendí los conceptos básicos del lenguaje de programación Go, donde aprendí a usar estructuras, funciones para abrir archivos, Imprimir resultados por terminal entre otras cosas

Imprimir Hello World

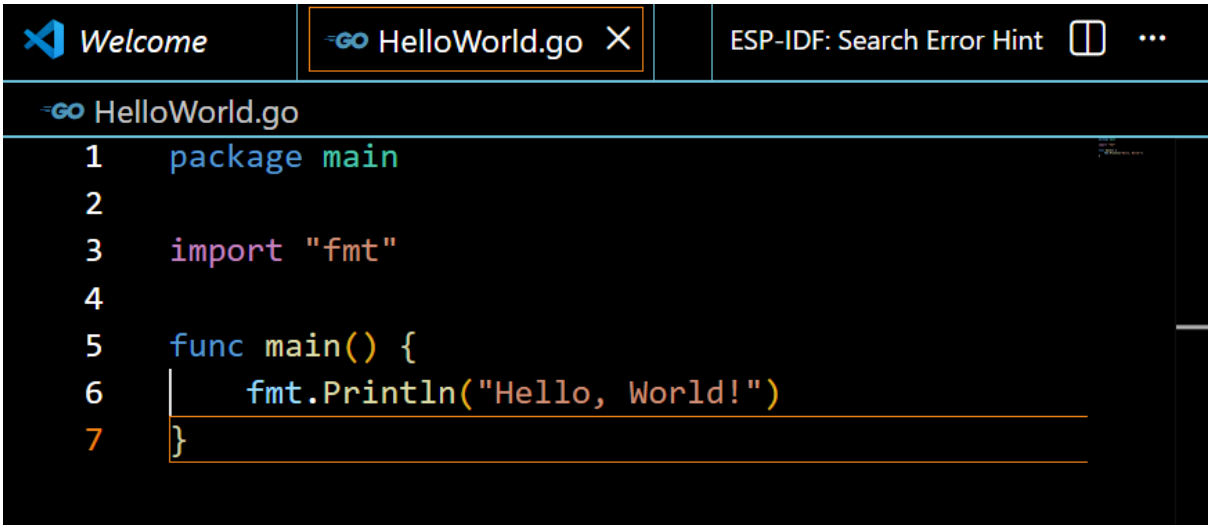

Figura 1. Hello World

`package main`: Le dice a Go que este archivo debe compilarse como un programa ejecutable, no como una librería.
`import "fmt"`: Importa el paquete "format", que contiene las funciones para imprimir texto en la consola.
`func main()`: Es el punto de entrada. Todo lo que pongas dentro de estas llaves `{ }` es lo que se ejecutará primero.

Figura 2. Creación de .exe

Build crea un archivo ejecutable (.exe) que será el archivo que ejecutaremos para activar lo que escribimos en el archivo .go. El último comando de la terminal lo ejecuta

Truncar un número decimal al entero más cercano

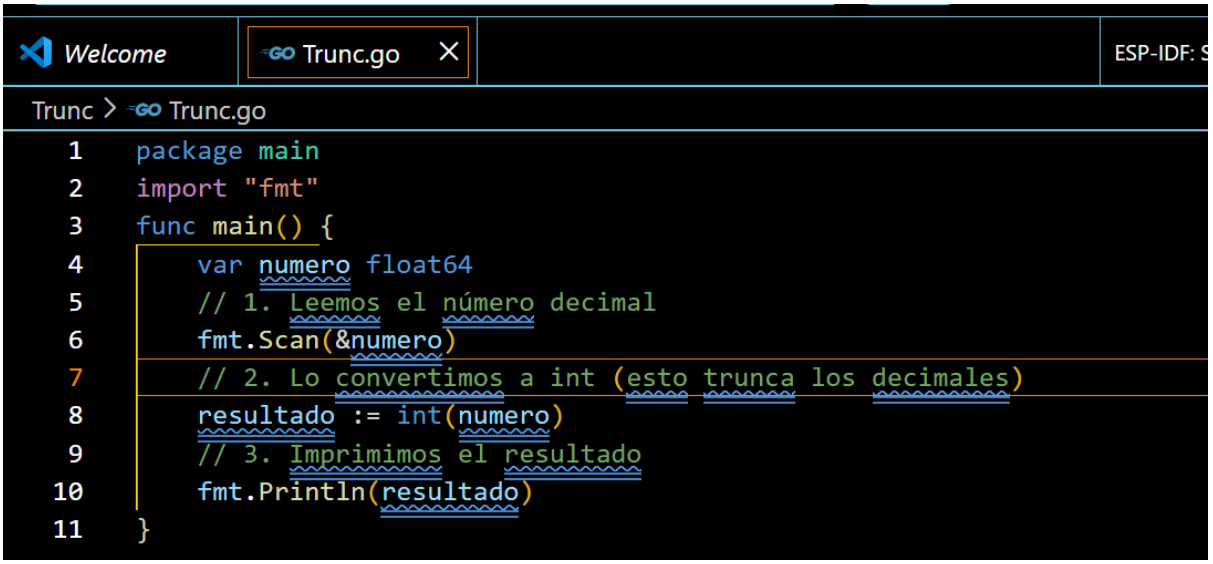

Figura 3. Código de truncar a entero

Creamos el nuevo archivo escribimos el codigo que nos solicitaron

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\Trunc> go build Trunc.go
```

Figura 3. Creación de ejecutable

creamos el ejecutable

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\Trunc> ./Trunc
12.5
12
```

Figura 4. Ejecución

Ejecutamos el .exe, tecleamos un número y lo trunca a entero más cercano

Find String

Este código tiene la función de pedir un texto por consola, cuando se ingrese el texto validará mediante condiciones si encuentra las letras especificadas entre las comillas si encuentra las tres, muestra encontrado, si falta alguna mostrará no encontrado

```
Trunc.go    Trunc.exe    FindString.go ×    ESP-IDF: Search Error Hint
FindString > FindString.go
1  package main
2  import (
3      "fmt"
4      "strings"
5  )
6  func main() {
7      var texto string
8      // El sistema ingresará la cadena aquí
9      fmt.Scan(&texto)
10     // Convertimos a minúsculas para asegurar que encuentre 'i', 'a', 'n'
11     texto = strings.ToLower(texto)
12     // Verificamos la presencia de las tres letras
13     if strings.Contains(texto, "i") && strings.Contains(texto, "a") && strings.Contains(texto, "n") {
14         fmt.Println("¡Encontrado!")
15     } else {
16         fmt.Println("¡No encontrado!")
17     }
18 }
```

Figura 5. Código

Creamos el código que se encargará de encontrar las letras que le indicamos en la condición if que letras son las que queremos filtrar para detectar si se encontró o no

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\FindString> go build FindString.go
```

Figura 6. Creación de ejecutable

Creamos el archivo ejecutable

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\FindString> ./FindString
gian
¡Encontrado!
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\FindString> ./FindString
dana
¡No encontrado!
```

Figura 7. Funcionalidad

Lo ejecutamos y verificamos funcionalidad

Slice

Este código pide números por terminal, los cuales almacena en Slice o “rebanadas” que seria como los arrays de otro lenguaje, para ordenar los numeros ingresados en el Slice

```go
package main
import (
    "fmt"
    "sort"
    "strconv"
)
func main() {
    // Creamos una rebanada vacía de enteros
    rebanada := make([]int, 0)
    for {
        var entrada string
        fmt.Scan(&entrada)
        // Si el usuario escribe 'X' o algo que no es número, el curso suele terminar
        // Pero aqui simplemente convertimos la entrada a entero
        numero, err := strconv.Atoi(entrada)
        if err != nil {
            break
        }
        // Añadimos el número a la rebanada
        rebanada = append(rebanada, numero)
        // Ordenamos la rebanada de menor a mayor
        sort.Ints(rebanada)
        // Imprimimos la rebanada ordenada
        fmt.Println(rebanada)
    }
}
```

Figura 8. Código

Creamos el código para almacenar en una rebanada o Slice, luego se ordenarán

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\Slices y Sort> go build Slice.go
```

Figura 9. Ejecutable

Creamos el archivo ejecutable

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\Slices y Sort> ./Slice
3
[3]
2
[2 3]
6
[2 3 6]
7
[2 3 6 7]
8
[2 3 6 7 8]
1
[1 2 3 6 7 8]
```

Figura 10. Resultado

Ejecutamos y comprobamos que los números ingresados los ordena y guarda en el Slice

JSON de ordenamiento

Código para crear un JSON que según lo ingresado en terminal mostrará un JSON ordenado con la estructura para identificar un mensaje de buena manera

```go
package main
import (
    "encoding/json"
    "fmt"
)
// Definimos la estructura del objeto que queremos crear
type Persona struct {
    Nombre    string `json:"nombre"`
    Direccion string `json:"direccion"`
}
func main() {
    var n, d string
    // 1. Leemos el nombre y la dirección
    // El sistema suele introducir los datos uno tras otro
    fmt.Scan(&n)
    fmt.Scan(&d)
    // 2. Creamos una instancia de nuestra estructura con los datos recibidos
    p := Persona{Nombre: n, Direccion: d}
    // 3. Convertimos la estructura a formato JSON (Marshalling)
    datosJSON, err := json.Marshal(p)
    if err != nil {
        return
    }
    // 4. Imprimimos el objeto JSON como una cadena de texto
    fmt.Println(string(datosJSON))
}
```

Figura 11. Código

Creamos el codigo para crear un JSON que devolvera en orden nombre y direccion

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\MakeJSON> go build MakeJSON.go
```

Figura 12. Ejecutable

Creamos el ejecutable

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\MakeJSON> ./MakeJSON
gian
tlaxcoapan hidalgo
{"nombre":"gian","direccion":"tlaxcoapan"}
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\MakeJSON>
```

Figura 13. Usabilidad

Lo ejecutamos y observamos que pidió un nombre, luego el lugar pero solo almacena sino escribimos espacios entre lo que digitalizamos en consola

Leer archivo y almacenar en estructuras

Este código se encargará de abrir un archivo de texto donde tendremos guardados nombres y apellidos como un alista, separado por un espacio como cualquier lista, el código va a pedir abrir el archivo de texto, luego leerá el texto linea por linea y detectara el nombre y apellido para guardarlo en una estructura separada el nombre del apellido

```go
package main
import (
    "bufio"
    "fmt"
    "os"
    "strings"
)
// Definimos la estructura con campos de 20 caracteres (en Go usamos string)
type Nombre struct {
    Fname string
    Lname string
}
func main() {
    var nombreArchivo string
    fmt.Print("Introduce el nombre del archivo: ")
    fmt.Scan(&nombreArchivo)
    // Abrimos el archivo
    archivo, err := os.Open(nombreArchivo)
    if err != nil {
        fmt.Println("Error al abrir el archivo:", err)
        return
    }
    defer archivo.Close()
    // Creamos la slice de structs
    personas := make([]Nombre, 0)
    // Leemos el archivo linea por linea
    scanner := bufio.NewScanner(archivo)
    for scanner.Scan() {
        linea := scanner.Text()
        partes := strings.Split(linea, " ")
        if len(partes) >= 2 {
            // Creamos el struct y limitamos a 20 caracteres si es necesario
            nuevaPersona := Nombre{
                Fname: partes[0],
                Lname: partes[1],
            }
            // Añadimos a la slice
            personas = append(personas, nuevaPersona)
        }
    }
    // Iteramos sobre la slice e imprimimos los resultados
    for _, p := range personas {
        fmt.Printf("Nombre: %s, Apellido: %s\n", p.Fname, p.Lname)
    }
}
```

Figura 14. Código

Creamos el archivo y escribimos el código

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\Read> go build Read.go
```

Figura 15. Ejecutable

Creamos el archivo ejecutable de nuestro codigo

```
Read > Nombres.txt
1   Juan Perez
2   Maria Garcia
3   Gian Arellano
4   Dana Castro
```

Figura 16. Lista de nombres

Creamos un archivo de texto donde guardamos nombres y apellidos como si fuera una lista

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\Read> .\Read.exe
Introduce el nombre del archivo:
```

Figura 17. Selección de archivo

Ejecutamos el archivo .exe y va a pedirnos el nombre del archivo que va a empezar a leer

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\Read> .\Read.exe
Introduce el nombre del archivo: Nombres.txt
Nombre: Juan, Apellido: Perez
Nombre: Maria, Apellido: Garcia
Nombre: Gian, Apellido: Arellano
Nombre: Dana, Apellido: Castro
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\Read>
```

Figura 18. Usabilidad

Le damos el nombre del archivo que contiene los nombres que creamos anteriormente y se verá como leyó el archivo y separó en dos estructuras el nombre y apellido

Bubble Sort

Esta práctica lo que hará es que pedira al usuario ingresar 10 números enteros para ordenarlos en un nuevo conjunto mostrando los numeros ingresados y como se ordenaron

```go
package main

import (
    "fmt"
)

// Swap intercambia el elemento en el índice i con el elemento en i+1
func Swap(numbers []int, i int) {
    // En Go podemos intercambiar valores en una sola línea
    numbers[i], numbers[i+1] = numbers[i+1], numbers[i]
}

// BubbleSort ordena la rebanada (slice) in-place de menor a mayor
func BubbleSort(numbers []int) {
    n := len(numbers)
    for i := 0; i < n; i++ {
        // El último i elementos ya están en su lugar
        for j := 0; j < n-i-1; j++ {
            if numbers[j] > numbers[j+1] {
                Swap(numbers, j)
            }
        }
    }
}

func main() {
    // Definimos un slice para guardar hasta 10 números
    var input int
    numbers := make([]int, 0, 10)
```

```
    fmt.Println("Ingrese hasta 10 números enteros (escriba un número no válido o alcance 10 para terminar):")

    for i := 0; i < 10; i++ {
        fmt.Printf("Número %d: ", i+1)
        _, err := fmt.Scan(&input)
        if err != nil {
            // Si el usuario ingresa algo que no es un número, dejamos de pedir
            break
        }
        numbers = append(numbers, input)
    }

    fmt.Println("\nLista original:", numbers)

    // Llamamos a la función de ordenamiento
    BubbleSort(numbers)

    fmt.Println("Lista ordenada:", numbers)
}
```

Crear el código para está práctica

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\Bubble Sort> go build BubbleSort.go
```

Crear el archivo ejecutable .exe para ejecutar nuestro programa

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\Bubble Sort> .\BubbleSort.exe
Ingrese hasta 10 números enteros (escriba un número no válido o alcance 10 para terminar):
Número 1: 2
Número 2: 1
Número 3: 3
Número 4: 5
Número 5: 7
Número 6: 4
Número 7: 2
Número 8: 1
Número 1: 2
Número 2: 1
Número 3: 3
Número 4: 5
Número 5: 7
Número 6: 4
Número 7: 2
Número 8: 1
Número 6: 4
Número 7: 2
Número 8: 1
Número 7: 2
Número 8: 1
Número 8: 1
Número 9: 2
Número 10: 1
```

Ejecutamos nuestro .exe y nos pedira ingresar los 10 números que ordenará

```
Lista original: [2 1 3 5 7 4 2 1 2 1]
Lista ordenada: [1 1 1 2 2 2 3 4 5 7]
```

Nos generará está salida con los números originales y los ya ordenados

GenDisplaceFn

Este programa utiliza **clausuras** (functions as values) para crear una fábrica de fórmulas físicas. El usuario ingresa la aceleración, velocidad y desplazamiento inicial, y el sistema genera una función personalizada que "recuerda" esos datos. Finalmente, al introducir un valor de tiempo esta función calcula y devuelve el desplazamiento final usando la fórmula cinemática.

```go
package main

import (
    "fmt"
    "math"
)

// GenDisplaceFn genera una función que calcula el desplazamiento
// Recibe: aceleración (a), velocidad inicial (vo) y desplazamiento
inicial (so)
```

```go
// Devuelve: una función que acepta el tiempo (t) y devuelve el
desplazamiento (s)
func GenDisplaceFn(a, vo, so float64) func(float64) float64 {
    // Retornamos una función anónima (clausura) que "recuerda" los
valores de a, vo y so
    return func(t float64) float64 {
        // Fórmula: s = 0.5 * a * t^2 + vo * t + so
        return (0.5 * a * math.Pow(t, 2)) + (vo * t) + so
    }
}

func main() {
    var a, vo, so, t float64

    fmt.Println("--- Calculador de Desplazamiento ---")

    // 1. Pedir valores iniciales al usuario
    fmt.Print("Ingrese la aceleración (a): ")
    fmt.Scan(&a)
    fmt.Print("Ingrese la velocidad inicial (vo): ")
    fmt.Scan(&vo)
    fmt.Print("Ingrese el desplazamiento inicial (so): ")
    fmt.Scan(&so)

    // 2. Generar la función de desplazamiento usando los valores
ingresados
    fn := GenDisplaceFn(a, vo, so)

    // 3. Pedir el tiempo (t)
    fmt.Print("Ingrese el tiempo (t) para calcular el desplazamiento:
")
    fmt.Scan(&t)

    // 4. Calcular y mostrar el resultado usando la función generada
    fmt.Printf("\nEl desplazamiento final tras %.2f segundos es:
%.2f\n", t, fn(t))
}
```

Creamos el código

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\GenDisplaceFn> go build GenDisplaceFn.go
```

Generamos el archivo ejecutable

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\GenDisplaceFn> .\GenDisplaceFn.exe
--- Calculador de Desplazamiento ---
Ingrese la aceleración (a): 10
Ingrese la velocidad inicial (vo): 2
Ingrese el desplazamiento inicial (so): 1
Ingrese el tiempo (t) para calcular el desplazamiento: 3

El desplazamiento final tras 3.00 segundos es: 52.00
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\GenDisplaceFn>
```

Ejecutamos el programa e ingresamos los valores solicitados para que el programa realice los cálculos y muestra el resultado.

Animals Structs and Methods

El programa organiza la información de tres animales (vaca, pájaro y serpiente) dentro de un `struct` llamado `Animal`, el cual contiene los campos para la comida, la locomoción y el sonido. Para cumplir con los requerimientos, define tres métodos: `Eat()`, `Move()` y `Speak()`, que utilizan a `Animal` como tipo receptor. Todo esto se gestiona mediante un mapa que actúa como base de datos y un bucle infinito que procesa la entrada del usuario, validando la petición y ejecutando el método correspondiente.

```go
package main

import (
    "bufio"
    "fmt"
    "os"
    "strings"
)

// Animal representa la estructura base con tres campos de tipo cadena
type Animal struct {
    food       string
    locomotion string
    noise      string
}

// Eat imprime la comida del animal
func (a Animal) Eat() {
    fmt.Println(a.food)
}

// Move imprime el método de locomoción
```

```go
func (a Animal) Move() {
    fmt.Println(a.locomotion)
}

// Speak imprime el sonido del animal
func (a Animal) Speak() {
    fmt.Println(a.noise)
}

func main() {
    // Inicializamos el mapa con los datos de la tabla proporcionada
    animals := map[string]Animal{
        "vaca":     {food: "hierba", locomotion: "caminar", noise:
"mugido"},
        "pájaro":   {food: "gusanos", locomotion: "volar", noise:
"pío"},
        "serpiente": {food: "ratones", locomotion: "deslizarse", noise:
"hsss"},
    }

    scanner := bufio.NewScanner(os.Stdin)
    fmt.Println("Interfaz de información de animales")
    fmt.Println("Instrucciones: Escribe el animal y la propiedad
(ejemplo: vaca comer)")

    for {
        fmt.Print("> ")
        if !scanner.Scan() {
            break
        }

        // Separamos la línea en palabras
        input := strings.Fields(strings.ToLower(scanner.Text()))

        // Validamos que se ingresen exactamente 2 cadenas
        if len(input) != 2 {
            fmt.Println("Error: Debes ingresar exactamente el nombre
del animal y la acción.")
            continue
        }

        name, action := input[0], input[1]
        animal, exists := animals[name]
```

```
        if !exists {
            fmt.Println("Error: Animal no encontrado (vaca, pájaro,
serpiente).")
            continue
        }

        // Ejecutamos el método correspondiente
        switch action {
        case "comer":
            animal.Eat()
        case "moverse":
            animal.Move()
        case "hablar":
            animal.Speak()
        default:
            fmt.Println("Error: Acción no válida (comer, moverse,
hablar).")
        }
    }
}
```

Creamos el codigo

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\Presentations> cd ..
PS C:\Users\giana\OneDrive\Escritorio\Curso Go> cd .\AnimalStructure\
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\AnimalStructure> go build .\AnimalStructure.go
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\AnimalStructure>
```

Creamos el archivo ejecutable

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\AnimalStructure> .\AnimalStructure.exe
Interfaz de información de animales
Instrucciones: Escribe el animal y la propiedad (ejemplo: vaca comer)
> vaca comer
hierba
> serpiente comer
ratones
> pajaro comer
Error: Animal no encontrado (vaca, pájaro, serpiente).
> pájaro comer
gusanos
>
```

Ejecutar el archivo ejecutable .exe y verificar el funcionamiento

Animal Interface Factory

Este programa implementa un sistema dinámico de gestión de animales utilizando **interfaces** en Go para lograr polimorfismo. Permite al usuario crear instancias personalizadas de vacas, pájaros o serpientes mediante el comando `newanimal`, asignándoles un nombre único almacenado en un mapa. Cada tipo de animal es una estructura independiente que satisface la interfaz `Animal` al implementar los métodos `Eat()`, `Move()` y `Speak()`. El sistema incluye un motor de comandos interactivo que valida la entrada y recupera los objetos para ejecutar sus métodos específicos según la solicitud del usuario.

```go
package main

import (
    "bufio"
    "fmt"
    "os"
    "strings"
)

// Animal es la interfaz que describe las capacidades de un animal
type Animal interface {
    Eat()
    Move()
    Speak()
}

// Estructuras de datos para cada tipo de animal
type Cow struct{}
func (c Cow) Eat()   { fmt.Println("hierba") }
func (c Cow) Move()  { fmt.Println("caminar") }
func (c Cow) Speak() { fmt.Println("mugido") }

type Bird struct{}
func (b Bird) Eat()   { fmt.Println("gusanos") }
func (b Bird) Move()  { fmt.Println("volar") }
func (b Bird) Speak() { fmt.Println("pío") }

type Snake struct{}
func (s Snake) Eat()   { fmt.Println("ratones") }
func (s Snake) Move()  { fmt.Println("deslizarse") }
func (s Snake) Speak() { fmt.Println("hsss") }
```

```go
func main() {
    // Mapa para almacenar los animales creados por el usuario
    animalMap := make(map[string]Animal)
    scanner := bufio.NewScanner(os.Stdin)

    fmt.Println("Bienvenido al Gestor de Animales")
    fmt.Println("Comandos: 'newanimal <nombre> <tipo>' o 'query
<nombre> <accion>'")

    for {
        fmt.Print("> ")
        if !scanner.Scan() {
            break
        }

        input := strings.Fields(strings.ToLower(scanner.Text()))
        if len(input) != 3 {
            fmt.Println("Error: Se requieren exactamente 3
parámetros.")
            continue
        }

        command, name, param := input[0], input[1], input[2]

        switch command {
        case "newanimal":
            // Lógica para crear un nuevo animal según su tipo
            var newAni Animal
            switch param {
            case "vaca":
                newAni = Cow{}
            case "pájaro":
                newAni = Bird{}
            case "serpiente":
                newAni = Snake{}
            default:
                fmt.Println("Error: Tipo de animal inválido (vaca,
pájaro, serpiente).")
                continue
            }
            animalMap[name] = newAni
            fmt.Println("¡Creado!")
```

```
        case "query":
            // Lógica para consultar un animal existente
            ani, exists := animalMap[name]
            if !exists {
                fmt.Println("Error: El animal con ese nombre no
existe.")
                continue
            }

            switch param {
            case "comer":
                ani.Eat()
            case "moverse":
                ani.Move()
            case "hablar":
                ani.Speak()
            default:
                fmt.Println("Error: Acción inválida (comer, moverse,
hablar).")
            }

        default:
            fmt.Println("Error: Comando no reconocido (newanimal o
query).")
        }
    }
}
```

Crear el codigo

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go> cd .\animal_interface_factory\
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\animal_interface_factory> go build .\animal_interface_factory.go
```

Crear el archivo ejecutable

```
PS C:\Users\giana\OneDrive\Escritorio\Curso Go\animal_interface_factory> .\animal_interface_factory.exe
Bienvenido al Gestor de Animales
Comandos: 'newanimal <nombre> <tipo>' o 'query <nombre> <accion>'
> newanimal firulais perro
Error: Tipo de animal inválido (vaca, pájaro, serpiente).
> newanimal firulais vaca
¡Creado!
> newanimal lola vaca
¡Creado!
> newanimal polli pájaro
¡Creado!
> query firulais comer
hierba
> query lola comer
hierba
> query polli moverse
volar
>
```

Ejecutamos, nos pedirá guardar algunos animales, los guardamos y luego en una query le preguntamos acerca de lo que hemos guardado y lo asocia a lo relacionado al tipo de animal.